

Adversarial Prompt Discovery in Large Language Models

By: Jakob Garcia, Soren Abrams, and Luis Cruz Hernandez

Introduction

This project investigates whether the GPT-2, a 124-million parameter language model trained on pre-2019 web text can be made to generate character names from *The Mandalorian* – a series that postdates the training corpus. The five target keywords are “grogu”, “mando”, “kuiil”, “peli”, and “fennec”. Because GPT-2 was never exposed to these names as complete words during training, the task is non-trivial as simple semantically relevant prompts like “Star Wars”, or “baby yoda” fail to generate any of the targets. The key insight that makes the task tractable is that GPT-2’s byte pair encoding tokenizer decomposes these unseen words into subword pieces that do exist in the training vocabulary (i.e. grogu becoming “g”, “rog”, and “u”)—meaning that a carefully constructed prompt can potentially induce the model to predict those subword pieces in the correct sequence. We explore this through two approaches: manual prompt engineering, and an automated gradient guided discrete search algorithm. The automated search achieved a 20/20 success rate across all five targets with 100% prompt reproducibility, identifying effective prompts that manual exploration alone would have been unlikely to discover.

Initially, the raw substrings alone were not enough to produce the target results. For example, the prompt 'gr ogu' produced the output of 'o, oguo oguo, oguo oguo, oguo oguo, oguo oguo, ogu'. Now this output is somewhat nonsensical, however it is clear to see that there is a pattern of the last characters of the prompt being repeated multiple times. Through manual fiddling, it was found that GPT-2 paid a lot of attention to the tokens on the ends of the prompt and would often repeat those characters in a nonsensical prompt. Thus it is made possible to use substrings of

characters of the target word on the ends of the prompt in order to force GPT-2 to recreate the target word. There was interesting behavior for the importance of tokens between the ends of the prompt as well. For the most part, the middle tokens served to force GPT-2 to interpret the prompt as nonsensical and thus be prone to repeating characters of the prompt (sometimes without middle tokens, the output would be real human language). The most significant role the middle tokens played however, was to tune how and where characters appeared in the output. Using these principles and a lot of trial and error, prompts were manually fine tuned to reach all target words showing that this strategy is valid. In terms of success rate, each keyword individually took (approximately) less than 20 prompts in order to find success.

Character Prompting:

Target	Success	Prompts Tested	Best Prompt
Grogu	N	3	N/A
Mando	N	3	N/A
Kuiil	N	3	N/A
Peli	N	3	N/A
Fennec	N	3	N/A

Hypothesis:

For the character prompting, we assumed that by separating the target words into individualized characters and letters, that the patterns would make GPT-2 reconstruct the words back into the target token.

Example Prompts:

g-r-o-g-u spells
letters g r o g u
m-a-n-d-o means

GPT-2 Behavior:

GPT-2 failed because the model interpreted the prompts as natural language instructions rather than putting the letters together. It seems like the GPT-2 model interpreted the letters as definitions. The model didn't seem to understand how to break the letters apart to reconstruct them back into the actual word. So it failed all tests.

Context Prompting:

Target	Success	Prompts Tested	Best Prompt
Grogu	N	3	N/A
Mando	N	3	N/A
Kuul	N	3	N/A
Peli	N	3	N/A
Fennec	N	3	N/A

Hypothesis:

Before we began testing, we believed that the GPT-2 model would not strongly associate the given words as context to what we were relating to. This was because of our assumption that the GPT-2 model relies on next word prediction. Which would mean that it will struggle reading the whole phrase and using it for full context.

Example Prompts:

grogu:

“small green child”
“masked bounty hunter”
“desert mechanic woman”

mando:

"masked bounty hunter",
"wandering masked fighter",
"silent helmet warrior"

kuul:

"old wise mechanic",
"desert engineer elder",
"quiet old inventor"

GPT-2 Behavior:

During testing, we noticed that the model would generate generic stories, relative text, and even repeat certain phrases and words (specifically nouns) when generating the results. When trying to provide the model contextual words, it seems to heavily struggle to put the words together. This is because it looks like the GPT-2 model heavily relies on “next word” prediction.

The model failed at producing the correct results because similar/related words/phrases were not enough to allow the GPT-2 model to understand and produce the target names. For example, to

us; the phrase “masked bounty hunter” would heavily imply a relation to the mandalorian. However, the GPT-2 model continued to generate broad and descriptive language instead of producing the target token we were looking for.

These results imply that the GPT-2 model is not good at handling a context approach prompt. Since it relies heavily on next word prediction. It is unable to build the context of the phrase and produce the desired output/result we are looking for

Acronym Prompting:

Target	Success	Prompts Tested	Best Prompt
Grogu	N	3	N/A
Mando	N	3	N/A
Kuiil	N	3	N/A
Peli	N	3	N/A
Fennec	N	3	N/A

Hypothesis:

Before we tested, we hypothesized that the GPT-2 model would take each individual letter and reconstruct it to find the target word. It would take each letter as a token, and then when it processed all of them, it would reconstruct the word and find the target word.

Example Prompts:

“grogu”:

"G R O G U stands for",
"the initials G R O G U mean",
"G-R-O-G-U means"

"mando":

"M A N D O stands for",
"the initials M A N D O mean",
"M-A-N-D-O means"

"kuiil":

"K U I I L stands for",
"the initials K U I I L mean",
"K-U-I-I-L means"

GTP-2 Behavior:

The GPT-2 model generated fake acronym expressions, over explanatory sentences, and some grammatical issues when trying to identify/define the acronyms

Semantic Prompting:

Target	Success	Prompts Tested	Best Prompt
Grogu	N	5-10	"The Sardinian word for yellow is "
Mando	Y*	5-10	"Famous manddolinist"
Kuiil	N	5-10	"is kui sick? yes,"
Peli	Y*	5-10	"My favorite band is Led"
Fennec	N	5-10	"The fenec fox "

*succeeded in generating the target as a substring of a larger word, but not in generating the target as a standalone word.

Hypothesis:

These prompts attempt to use existing semantic associations to generate the desired sequence of characters. These words, while they are mostly nonsense words, do not exist in a vacuum. The words are similar to other words that do have well-known meanings. For example, the word “fennec” is heavily associated with foxes, due to a species of the same name, and “mando” can be described as a combination of the words “man” and “do.” Because of this, we hypothesized that forcing the model to generate these words – with some additional instructions for string manipulation if necessary – would effectively generate the target strings.

Example Prompts:

“grogu”:

"A mix of water and rum + u is" (trying to generate “grog” followed by “u”),
"Fast tempo marking + firearm is" (trying to generate “allegro” followed by “gun”),
"The Sardinian word for yellow is "

"mando”:

"German for commander is Kom" (trying to complete “Kommando”),
"Giuseppe Branzoli was " (trying to get the word “mandolinist”),
"Famous manddolinist" (trying to get the word “mandolin”)

"fennec”:

"The fox with big ears is called ",

"Vulpes zerda, commonly known as ",
"The fenec fox "

GTP-2 Behavior:

This prompting technique can work in certain circumstances, but is highly dependent on the desired keyword, the strength of the model, and the allotted number of tokens. With this prompting technique, 2/5 of the targets were able to be found, however, they were only found in the context of being substrings of larger words. Attempts to find them as standalone words all failed. This is mainly because the model was too obstinate to allow for complex semantic associations or string modifications. For example, for the target 'grogu,' initial tests focused on generating the alcoholic beverage "grog" followed by the character "u." However, none of the attempts focused on this strategy were able to yield results; even if the alcoholic beverage was generated – which did not happen once during testing – getting the model to generate a "u" any particular word proved difficult, even more difficult if accounting for spaces. Therefore, the successful attempts focused primarily on finding the targets as substrings of larger words. For the target 'peli,' I noticed that "zeppelin" contains the target as a substring, and that "Led Zeppelin" is a very famous band, so by simply getting the prompt to finish the band name, I was able to get the target as a substring. Likewise, I was able to get the target 'mando' by finding a prompt which generated "mandolin," which has the target as a substring. This was a bit trickier, but I found that by misspelling "mandolinist" in such a way that it no longer contained the substring "mando," the model still properly interpreted it as "mandolin musician." This technique failed for 'grogu,' 'kuiil,' and 'fennec.' This is because these substrings are very uncommon as substrings of English words, and even if one was found, the model likely would not know that word. For 'grogu,' I found via Wiktionary that "grogu" is the Sardinian word for "yellow," but it does not appear that the model has a sufficient Sardinian corpus for this technique to work. This problem would be true for the other meanings in other languages. This technique would likely be more effective on more advanced models with more data. For 'kuiil,' this has the same problem, with the added detriment of "kuiil" not being a word in any language. To this end, I attempted to craft a situation in which the person Kui has a sickness, attempting to leverage the fact that "kuiil" could be formed by combining the words "Kui" and "ill." However, even if I could force these tokens to appear, the amount of tokens allotted means that it would be exceedingly difficult to get the words to not only appear, but to appear immediately next to each other without a space in between. For the target 'fennec,' since "fennec" is not a substring of any English words, I attempted to force the model to generate something related to Fennec Foxes, an African fox species with distinctive large ears. However, the main hindrance for this strategy was that the model did not appear to know about this species of fox. Despite having several methods of precisely identifying Fennec Foxes in particular -- including the scientific name, characters based on the animal, features of the animal, and even using the same misspelling tactic as worked for "mandolin" -- the model would never generate "fennec" in relation to foxes. This would likely be

solved if we used a model with more data, as it would likely have data relating to Fennec Foxes in the corpus.

First Prompting Testing Notes:

- Manual testing for specific prompts. Decided to start with a smaller test sample to see the results. This made debugging easier and finding simpler solutions.
- Will be using a more dedicated sample for further analysis and testing of the code.

Results Expansion:

It is clear that manual prompting did not provide us with any relevant information regarding any character or context prompting. It was ineffective compared to the token-based approaches.

There might be some success that could be observed in substring/prefix strategies that are similar to the ones used in GPT-2's subword tokenization.

In my testing, GPT-2 tried to generate general descriptive text rather than actually producing the specific names that I was providing. Context prompting has failed across all of the targets. This can be seen in examples where when I gave prompts like “masked bounty hunter” and “green alien child”, GPT-2 just provided descriptive text instead of the names.

B) Test Harness:

In order to test the correctness of the prompts above, we implemented a validation function that checks for three main conditions. First, the keyword must not appear in the prompt at all—even as a substring of a larger string. For example, for the keyword “grogu” a prompt such as “abc**grogu**def” would not be considered correct as the keyword appears as a substring, however a prompt of “abc gro gu def” would be considered valid as the keyword is not contiguous.

The second condition is that prompt length must be less than or equal to the maximum number of allowed tokens when tokenized (the maximum number we used was 10). The key stipulation here is that we must account for how tokenizers are not invertible, meaning that when converting from tokens to a string, going back to tokens can produce different results and lengths since the string context can be interpreted differently.

The final condition is that the keyword must appear in the generated output as an independent word. This means that if the output were something like “hel**logrogu** there”, this output would not be accepted as the keyword only appears as a substring of a larger term. In order to be accepted, the keyword must be separated by any of the following: whitespace, the ends of the string, or punctuation characters.

The validation function can be viewed below:

```

def test_case(model, tokenizer, prompt, keyword, max_tokens=MAX_PROMPT_TOKENS):
    """
    Evaluate whether a given prompt satisfies all the assignment restrictions/requirements. And it will
    then successfully generate all the desired target keywords

    The prompt will either be a:
    - raw string input
    - a list of token ID's

    Restrictions:
    1. The keyword must NOT appear in the prompt
    2. The prompt must be <= max_tokens after the tokenization
    3. The keyword must appear as it's own word in the GPT-2 output.

    ...
    Returns:
    It will return a boolean if the test passed or failed, and then a string explaining why
    it passed or failed
    """
    # handle token ids and raw strings
    if type(prompt) == list:
        prompt = tokenizer.decode(prompt)

    # Condition 1: keyword not in prompt
    if keyword.lower() in prompt.lower():
        return (False, 1, f"FAIL [constraint 1]: prompt contains '{keyword}'")

    # Condition 2: token length — ALWAYS check re-tokenized length
    actual_length = len(tokenizer.encode(prompt))
    if actual_length > max_tokens:
        return (False, 2, f"FAIL [constraint 2]: {actual_length} tokens > {max_tokens}")

    # Condition 3: keyword in generated output
    output = generate(model, tokenizer, prompt)
    # outputted word should be standalone. e.x. "a grogu b" is good, "agrogub" is not
    if re.search(fr"\b{keyword.lower()}\b", output.lower()):
        return (True, 0, f"PASS: output = {repr(output)}")
    else:
        return (False, 3, f"FAIL [constraint 3]: output = {repr(output)}")

```

2. LLM Usage

For this project, Claude was the primary LLM that was used to assist with coding tasks. We opted to create a project in Claude and provide all the meaningful context about our task so that it had a better understanding of what we were trying to accomplish. Before beginning any coding, we made sure that the LLM was aligned with the task by having it explain the key concepts of the project and engage in planning out how we will begin to code this.

Overall, the LLM was good at creating the initial pass at the code, as sometimes the hardest thing is to know where to get started. Considering that the topic in general was something that was new to our whole group (especially how to do discrete gradient descent), using an LLM helped us complete this task faster. There were a couple of places where the LLM did struggle however, namely that the code it produced would often have parts that were unneeded, such as having extra variables that went unused or generally misinterpreting how certain Python constructs work. It was able to get the majority of the code correct on the first try, but an understanding of the code and what outputs should be was still very necessary in order to pick out smaller behavioral (non-runtime error) bugs that existed in the code. So it was primarily used to draft a function once, and then we fixed the function ourselves to make sure it fit within the larger scheme of the program and task requirements. For larger issues that the LLM missed, we would point out to the LLM what was wrong and then see if it could fix it. An important skill that was used was being able to read and truly understand code that you did not write yourself in order to be able to meaningfully contribute to it. A traditional skill that was not practiced was spending the time to figure out the initial structure of the program.

I would estimate that about 75% of the code was written by AI, and 25% was written ourselves. Now of the 75%, we then manually edited about 5-10% to fix small behavior bugs that the LLM missed on its first pass. Examples of LLM prompts and outputs are shown below.

Example of the LLM being incorrect

Python's `in` operator checks for substring presence — so `"grogu" in "alpgrogub"` is actually `False` here because `"grogu"` doesn't appear as a contiguous substring in `"alpgrogub"`. Let me verify the exact behavior:

```
python
"grogu" in "alpgrogub"    # → False ✓ (passes constraint)
"grogu" in "ab grogu cd"  # → True  ✓ (fails constraint)
"grogu" in "xgrogu"       # → True  ✓ (fails constraint — "grogu" IS a substring)
"grogu" in "GROGU"        # → False without .lower(), True with .lower()
```

This is an incorrect statement because Python would return `True` for that statement, meaning that it also made up the results of the Python cell.

```
>>> "grogu" in "alpgrogub" # → False ✓ (passes constraint)
True
```

Example of getting the LLM to create a first pass at the discrete gradient descent search
The prompt is shown below:

* Prompt before this asks the LLM to give a background knowledge crash course on GPT-2, BPE, and embeddings. *

Let's discuss the automated search algorithms. I think a discrete gradient descent approach would be interesting to start with since we can directly deal with the encoded tokens of strings.

Great choice. Gradient-guided discrete search is the most principled approach here and directly connects to how modern adversarial NLP research works (it's the core idea behind GCG — Greedy Coordinate Gradient — from the "Universal and Transferable Adversarial Attacks" paper). Let me build this up carefully.

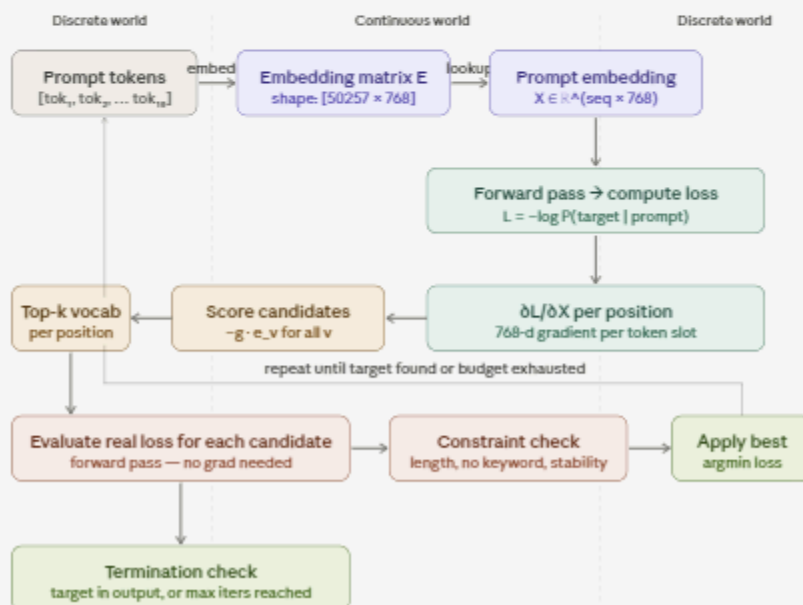
The Core Idea: Why Gradients on Discrete Tokens?

Here's the tension you need to understand first:

Gradients require continuous math. Calculus works on smooth, differentiable functions. You can ask "if I nudge this value slightly, how does the loss change?" — that's what a gradient is.

Token IDs are discrete integers. Token 4321 and token 4322 have no meaningful mathematical relationship. There's no "slightly" between them. You can't compute $\partial \text{loss} / \partial \text{token_id}$ in any useful sense.

So the trick is: **don't take gradients with respect to token IDs. Take them with respect to the token embeddings** — the continuous 768-dimensional vectors that represent each token.



Now let's go through each step carefully with the actual math and code.

It then wrote out 5 functions. In order to avoid taking up too much space, I will list the descriptions of each step instead. The full functions can be viewed in the project repository.

Step 1 — The objective function

You need a scalar number that measures "how bad is this prompt at generating the target?" That's your loss. The natural choice is negative log probability: $L = -\log P(\text{grogu} \mid \text{prompt})$

```
> def compute_loss
```

Step 2 — Computing the gradient

This is the key step that separates gradient-guided search from brute-force. Instead of asking "what if I randomly replace this token?", we ask "in which direction should I move in the embedding space to decrease the loss?"

```
> def compute_token_gradients
```

Step 3 — From gradient to candidate tokens

Here's the clever part. The gradient g at position i is a 768-dimensional vector that tells you: "if you move the embedding at position i in direction $-g$, the loss decreases fastest."

But you can't set an embedding to an arbitrary point — you can only use real vocabulary tokens. So you ask: which vocabulary token's embedding is most aligned with the direction of steepest descent?

Mathematically, you want the token v that maximizes $-g \cdot e_v$ (dot product), which is the same as finding the vocabulary embedding most similar to $-g$.

```
> def get_top_k_candidates
```

Step 4 — Verify with actual forward passes

The gradient approximation is just a filter — it narrows 50,257 candidates down to ~20 promising ones. You still need to actually compute the real loss for each candidate to find the best one:

```
> def find_best_replacement
```

Step 5 — The outer search loop

Now wrap everything into the iterative search:

```
> def gradient_guided_search
```

Example of asking the LLM to correct an algorithmic bug

The prompt is shown below:

For this function you wrote earlier:

* reference to a previous code cell for gradient-guided discrete search *

I think there is some logic we should work out on randomly initializing if it gets stuck. Mainly, I think there should be some kind of tolerance factor when restarting, as currently it gets stuck in a loop when restarting since it restarts, and then checks if the new random position is better than the best loss which usually is no since it didn't get to explore yet, and then it randomly selects another prompt. Essentially, this is just being a slightly smarter random search in this case. Should we either reset the best loss when restarting, or allow for a couple iterations of exploration before restarting?

The output:

Great catch — this is a real algorithmic bug, not just a style issue. Let me reason through it carefully.

What's actually happening

The current logic has a subtle conflict between two things that should be independent: the **global best loss** (tracking the best prompt seen across the entire search) and the **local exploration loss** (deciding whether the current prompt is making progress). Right now they're the same variable, which creates the trap you identified:

1. Search finds a decent prompt with loss = 0.8
2. Gets stuck → restarts with random prompt, which has loss = 3.4
3. Explores for a few iterations, gets to loss = 1.2 (genuine improvement from that start)
4. But $1.2 > 0.8$, so `improved = False` → immediately restarts again
5. Repeat forever — you're just doing random search with extra steps

The core issue is that "no improvement" means two very different things depending on context:

- During normal search: "I've exhausted this neighborhood, time to escape"
- After a restart: "I haven't beaten my all-time best yet" — which is almost always true immediately after a restart, and is the wrong signal entirely

So the question is: which fix is right? Let me think through both options.

Option A — Reset best loss on restart. Simple, but it throws away information. If your best-ever loss was 0.3 and you restart, you might wander for many iterations in a much worse region of the landscape before the loss signal tells you to move on. You lose the ability to compare "is this new starting region any good at all?"

Option B — Allow N exploration iterations before checking improvement. This is the right framing. You're decoupling two separate concerns: "give this starting point a fair chance to develop" vs. "decide whether to abandon it." This is essentially a patience/tolerance parameter — a standard technique in ML training (early stopping uses the same idea).

Option B is the right call. Here's the key insight: you actually want to track two loss values simultaneously — the global best (for bookkeeping and final reporting) and a local baseline (for deciding whether the current starting point is worth continuing from).

The fix

python

```
def gradient_guided_search(model, tokenizer, target, max_iters=200, k=20,
                           seed=42, patience=5):
    """
    patience: how many full iterations without local improvement
              before triggering a restart. Give each starting point
              a fair chance to explore before giving up on it.
    """
    torch.manual_seed(seed)
    random.seed(seed)

    prompt_ids = initialize_random_prompt(tokenizer, target, max_tokens=10)

    # --- Two separate loss trackers ---
    global_best_loss = float('inf') # best seen across ALL restarts - for reporting
    global_best_prompt = None

    local_best_loss = float('inf') # best seen since last restart - for restart logic
    iters_without_local_improvement = 0

    for iteration in range(max_iters):
        improved_locally = False

        for pos in range(len(prompt_ids)):
            new_token, new_loss = find_best_replacement(
                model, tokenizer, prompt_ids, target, pos, k=k
            )

            if new_loss < local_best_loss:
                prompt_ids[pos] = new_token
                local_best_loss = new_loss
                improved_locally = True

            # Also update global best if this is an all-time best
            if new_loss < global_best_loss:
                global_best_loss = new_loss
                global_best_prompt = tokenizer.decode(prompt_ids)

        # Check for success
        decoded = tokenizer.decode(prompt_ids)
        passed, _ = test_case(model, tokenizer, decoded, target)
        if passed:
            print(f" Found at iteration {iteration}: '{decoded}'")
            return decoded

    if iteration % 10 == 0:
```

3. Automated Prompt Search

The gradient guided discrete search algorithm was run 20 independent times per target keyword, each starting from a different random prompt seed. The first table below summarizes the results. The second table below shows comparisons against the different baselines.

Automated Search Results Across 20 Runs per Target

Target	Success Rate	Prompts Tested	Time	Best Prompt
Grogu	20/20	266	81m 47.7s	' nicRoguebuutm matt Gund Gw forgiven boutique nut'
Mando	20/20	47	12m 17.7s	'mandiflower Spo asphaltRepeMand indo Nicaragua comprom pronounce'
Kuul	20/20	193	53m 8.2s	'abaseoluuuul Utilities UI UnKal var ve'
Peli	20/20	109	34m 2.6s	' populous BMCpelhei?????-?????- Am pelPal pronunciation Ves'
Fennec	20/20	78	19m 54.9s	' Blasio TradablefeTraEStream Quadrojohnser Tradable Pound'

Prompt Search Method Comparisons

Method	Success Rate	Prompts Tested	Avg. Time
Random Baseline	0%	2500/2500	~10 minutes per keyword
Manual	Very low	~200	< 1 minute per prompt
Gradient Guided Search	100%	693/10000	~40 minutes per keyword

The table above compares the gradient guided search against the two baselines of random search and the manual prompts from Section 1. The random baseline generated random valid prompts (subject to the 10-token budget and no-keyword constraints) and tested each against the target. This was repeated for up to 100 attempts per run across 5 runs per keyword (approximately 10 minutes of compute per target) with zero successes across all five targets. This result is unsurprising as the probability that a random 10-token prompt happens to produce a specific rare subword sequence is very small.

The manual prompts from Section 1 present a different profile. The substring strategy succeeded on all five targets within approximately 20 attempts each, demonstrating that human

intuition about GPT-2's subword repetition behavior can be highly effective. However, manual prompting required direct knowledge of the target's tokenization structure, as a person had to reason about which character fragments to use. The semantic, context, and phonetic strategies largely failed, succeeding on at most 2/5 targets and only as substring matches within larger words. The gradient-guided search, by contrast, achieved a 20/20 success rate with full word matches across every run, and did so without any human knowledge of the target's internal structure—the algorithm discovered effective prompts automatically from gradient signal alone.

Analysis

We chose to use the gradient guided discrete search approach because the search space is inherently discrete (token IDs are integers with no meaningful ordering) but GPT-2's internals are continuous and differentiable. Purely discrete methods such as the random baseline and greedy random search must evaluate each candidate prompt with a full forward pass, making exhaustive searches over the 50,257 token vocabulary. The gradient-guided approach resolves this by operating in the continuous embedding space, computing the loss for each prompt position, then using the dot product between the gradient and each vocabulary token's embedding vector as a cheap linear approximation of which token replacements will reduce loss. This narrows 50,257 candidates down to a top-k shortlist ($k=20$ in our implementation) that are then verified with actual forward passes, combining the efficiency of gradient information with the accuracy of direct loss evaluation.

The gradient-guided search found valid prompts in an average of 6.93 candidate prompts per success (693 total prompts across 20 runs times 5 targets, over 100 total successes). This is orders of magnitude more efficient than the random baseline, which produced zero successes in 100 attempts per run. The computational cost is dominated by the gradient computation at each iteration, as for each token position, one backward pass through GPT-2 is required to obtain a gradient, followed by up to $k=20$ forward passes to verify candidates. With a prompt length of 10 tokens and our patience budget of 5 iterations before restart, the worst-case per-iteration cost is 10 backward passes + 200 forward passes, which accounts for the observed wall-clock times of 12–82 minutes per target across 20 runs.

An interesting pattern noticed in the successful prompts found by the search algorithm is that they seem to fall inline with the same idea as the substring prompting strategy, where the characters that make up the target word are littered in the prompt. Some examples are shown below:

Grogu

- 'Rogue muzzleUGu..... Gothic Geh Rogue discontin hy'
- 'stonesaceae 660gurRogueu\rogu 840'

Mando

- 'mandño76561 mascara mand Operationurally Goat mandatoryorsche'
- 'geryazakiimo Mand Omega anniversary BeiaborIGalle'

Kuiil

- '=akis KiwUIilituriil Learnsdk es'
- 'Neph cul QueenslandIlv Kiwuiil thous Sik'

Peli

- 'pelgiPalovi PAL pelvicugipel◆ple'
- 'Crafting massageる Pelhei pel Ai knitting PelMic'

Fennec

- 'FernfoxfenRIPT Tammyapego amd Palo amd Jenner'
- 'ennec Companーテenne◆◆◆◆\FelOtt Cele'

As you can see, many of the prompts are filled with random assortments of characters and words, but they all contain substrings of the target word in various places. You can even see related words being used in the prompts such as “fox” for fennec. Something that was a bit surprising to see was all different symbol/punctuation characters, as well as different alphabet systems like Japanese appearing in the successful prompts. It makes one ponder what kind of signal was being received by these characters in the gradient descent.

In terms of future improvements, there are several directions that could meaningfully improve on the current approach if given more time. For starters, the current loss function sums cross entropy independently over each target token position. For targets like “kuiil”, this treats the three tokens as independent objectives, but in practice the prompt needs to create a context that primes all three sequentially. A loss formulation that explicitly rewards joint probabilities may better capture the sequential dependencies. Additionally, the way the gradient search is initialized could be improved, as currently it is using random tokens. This could be improved by initializing the gradient search from successful manual prompts rather than random tokens, so that we could combine human intuition about semantic neighborhoods with the algorithm’s ability to fine-tune at the token level, potentially reducing search time for targets with identifiable semantic associations.

4. Error Analysis

Target	Category	Average Reproduction Rate	Total Constraint 1 Violations	Total Constraint 2 Violations	Total Constraint 3 Violations	Total Tokens Evaluated
Grogu	Complete Success	100%	234	4160	248	53200
Mando	Complete Success	100%	154	628	27	9400
Kuul	Complete Success	100%	411	2884	174	38600
Peli	Complete Success	100%	566	1361	89	21800
Fennec	Complete Success	100%	0	1547	58	15600

Constraint Violations

As described in the **Test Harness** section, there are three conditions that the automated prompt searching has to abide by. The primary constraint for scientific validity is checking if the keyword is in the prompt. Without it, trivial solutions like “say the word grogu” could satisfy the task while revealing nothing about GPT-2's internal representations. The constraint forces the search to find prompts that produce the target through distributional context rather than direct copying, which is what makes a successful prompt scientifically interesting.

What would constitute cheating this constraint would be any prompt containing the target as a contiguous substring, case-insensitively. For example, “please say grogu now” or “xGROGUy” would both violate this constraint. Our implementation checks for the keyword in the prompt and catches all contiguous substring occurrences regardless of surrounding characters or capitalization.

Over the course of the 20 searches for each keyword, the number of constraints violated were tracked, as well as the number of tokens evaluated when trying to find a better prompt. The number of tokens evaluated is not the same as the number of prompts tried, but instead the number of tokens that were being tested while building new prompts. The full results can be seen in the table above, but the most interesting results were that the keyword “fennec” had 0 violations of the first constraint across 15,600 candidates evaluated, while the keyword “peli” had 566 violations of the first violation across 21,800 candidates.

For the second constraint of the prompt not exceeding 10 tokens, the 10 token budget ensures comparability across search methods and prevents trivially long context windows from

providing too much signal. The re-tokenization requirement (decoding to a string and then re-encoding) is critical, as two tokens that decode to adjacent characters may re-tokenize as a single token, and vice versa—so counting tokens in the original candidate list without re-tokenizing would produce prompts that silently violate the budget at test time. Constraint 2 is also the dominant source of violations across all targets, accounting for 84% of all violations when summed across the results. The high constraint rate reflects how gradients prefer to point towards long, information-dense tokens.

The final constraint is simply checking whether a prompt resulting from gradient descent produces the target output or not. Compared to the other constraints which are primarily checked while building a new prompt, this final constraint is one that checks the prompt as a whole for correctness and indicates if more searching needs to be done. The total counts of this constraint being violated are very close to each of the number of prompts tested, indicating that once a prompt had finished undergoing gradient descent, it was unlikely to violate the other constraints.

Pattern Identification

In order to identify behavior in the difficulty of generating each keyword, it is important to understand how each target gets tokenized:

grog	→	[70, 3828, 84]	→	['g', 'rog', 'u']
mando	→	[22249, 78]	→	['mand', 'o']
kuil	→	[74, 9019, 346]	→	['k', 'ui', 'il']
pel	→	[30242, 72]	→	['pel', 'i']
fennec	→	[69, 29727, 66]	→	['f', 'enne', 'c']

The easiest keyword to generate was “mando”, and this can be attributed to the fact that it is only 2 tokens. More importantly, “mando” overlaps with the English language a lot, appearing in words like “**commando**” and “**mandolin**”. This means GPT-2’s training corpus contained meaningful co-occurrence statistics for this token in multiple contexts, giving the gradient clear signal about which prompt tokens increase its probability.

The second easiest keyword was “fennec”, which was made up of 3 tokens. Despite 3 tokens being harder to output (as seen in “grog” and “kuil”), fennec was much easier to generate, as fennec is a real word (i.e. fennec fox) and thus has more real contexts that it can occur in.

The target that lies in the middle of the difficulties was the keyword “pel”, which broke into the tokens “pel” and “i”. Both of these tokens can appear in a variety of contexts. The single character of “i” can be used to make up other words or be a standalone singular pronoun appearing in many texts. But both of the tokens also appear together in many words, giving a precedent for being able to put the two tokens together. Words like “**pelican**”, “**pipeline**”, and “**zeppelin**” are all examples of this. Perhaps the frequency of this substring is the cause of the high amount of constraint 1 violations for this keyword.

The second hardest keyword was “kuil”, which splits into 3 tokens. In general, the “iil” subword is particularly unusual, as double “i” sequences are rare in English. GPT-2’s training

data would have limited co-occurrence statistics for “ui” and “il” as a sequential prediction chain, requiring the search to find a prompt that creates a very specific and uncommon distributional context. The high constraint 2 violation rate (2,884) indicates the gradient was frequently pointing toward longer tokens, which can be a sign that the search was struggling to find valid candidates at multiple positions simultaneously.

The hardest keyword was “grogu”, which also splits into 3 tokens. The difficulty likely stems from the distributional ambiguity of the tokens, as there are no English words that contain the substring “grogu”. We can find similar words such as “**rogue**” or “**groggy**”, but the aspect of having “rog” both start with a “g” and end with a “u” is harder to find, as the gradient must overcome a strong prior that these characters don’t typically go together.

Outcome Categorization

All five targets achieved “Complete Success” status under our evaluation criteria of 20/20 search success rate and 100% prompt reproducibility over 10 trials using word-boundary output matching. This is a stronger result than anticipated given the diversity of the targets.